

PostgreSQL Scaling Lab

Deep Dive: Indexing, Partitioning, and Read Replicas

Explainer document for Salman Adnan's portfolio project `postgres-scaling-lab`

Generated July 7, 2026

What this document is: an exhaustive, beginner-safe walkthrough of every part of this project, generated by reading the actual source files, SQL, configuration, and README/answer documents in the repository. Nothing here is invented: numbers, behaviors, and code excerpts are taken directly from the project. Anything that could not be confirmed from the code is explicitly flagged as *inferred* or *not verified*.

Contents

1	What this project is	3
1.1	Authorship, honestly	3
1.2	Repository layout	4
2	Part 1: Slow-query logging and indexing	5
2.1	The scenario	5
2.2	Finding the slow queries: <code>slow-query logging and auto_explain</code>	5
2.3	What an index actually is	6
2.4	The solution: <code>part_1/solution.sql</code>	7
2.5	Measured results	8
3	Part 2: Range partitioning for time-series data	9
3.1	Why indexing alone stops helping	9
3.2	What table partitioning is	9
3.3	Mixed-granularity partitioning strategy	10
3.4	Boundary semantics	11
3.5	Measured results, including the honest negative one	11
3.6	A known limitation left in the code, disclosed rather than hidden	12
4	Part 3: Primary plus two streaming read replicas	13
4.1	The remaining bottleneck	13
4.2	WAL and streaming replication	13
4.3	Cluster topology	13
4.4	Configuration walkthrough	14
4.4.1	A committed secret, found and fixed	14
4.5	The routing application: <code>app.py</code> versus <code>solution.py</code>	15
4.6	Replication lag and the “read-your-writes” problem	16
4.7	Test suite and what it demonstrated	16

5	Verification status — what was actually run, stated plainly	18
6	Challenges encountered, in the author's own words	18
7	What was learned	19
8	What the author would do differently	19
9	Appendix: file-by-file quick reference	20

1 What this project is

This is a three-part, hands-on study of how far a single PostgreSQL database can be pushed before reaching for a different kind of database system entirely. It was built solo by Salman Adnan for an Enterprise Software Development course, against course-provided scaffolding (the assignment briefs, base schemas, a load-testing tool, and a baseline application). The premise, taken directly from the assignment brief (`docs/ASSIGNMENT.md`), is a workplace scenario: a company runs everything on PostgreSQL, queries have started getting slow, and the CTO wants to jump straight to a NoSQL database (like MongoDB or Cassandra) because “NoSQL is infinitely scalable.” The project’s answer is: don’t jump yet — there is a large amount of performance still available in plain PostgreSQL using its standard toolbox, and the project demonstrates three of those tools in order, each solving the specific problem the previous one left behind.

RDBMS (Relational Database Management System) is a database that stores data in tables with rows and columns and enforces relationships between them; PostgreSQL is one such system. **ACID** (Atomicity, Consistency, Isolation, Durability) is the set of guarantees an RDBMS like PostgreSQL gives about transactions: they either fully happen or not at all, the data is always left in a valid state, concurrent transactions do not corrupt each other, and committed data survives crashes. NoSQL databases often trade some of these guarantees for easier horizontal scaling; the project’s thesis is that you should only make that trade once you have actually exhausted what PostgreSQL itself can do.

The three techniques, applied in sequence, each attack a different bottleneck:

- Part 1. Indexing** — some individual queries are slow because PostgreSQL has to read every row of a table to answer them. Targeted indexes let it jump straight to the matching rows instead.
- Part 2. Partitioning** — some queries are slow even with indexes, because the table itself (and its indexes) are too large and mix “hot” recent data with “cold” historical data in the same structures. Splitting one big table into many smaller physical tables lets PostgreSQL skip whole chunks of data it knows cannot match.
- Part 3. Read replicas** — even a perfectly indexed, perfectly partitioned table is still served by one machine. If most of the traffic is reads, copying the data to additional read-only servers and spreading read queries across them relieves the single server of that load, without touching how writes work.

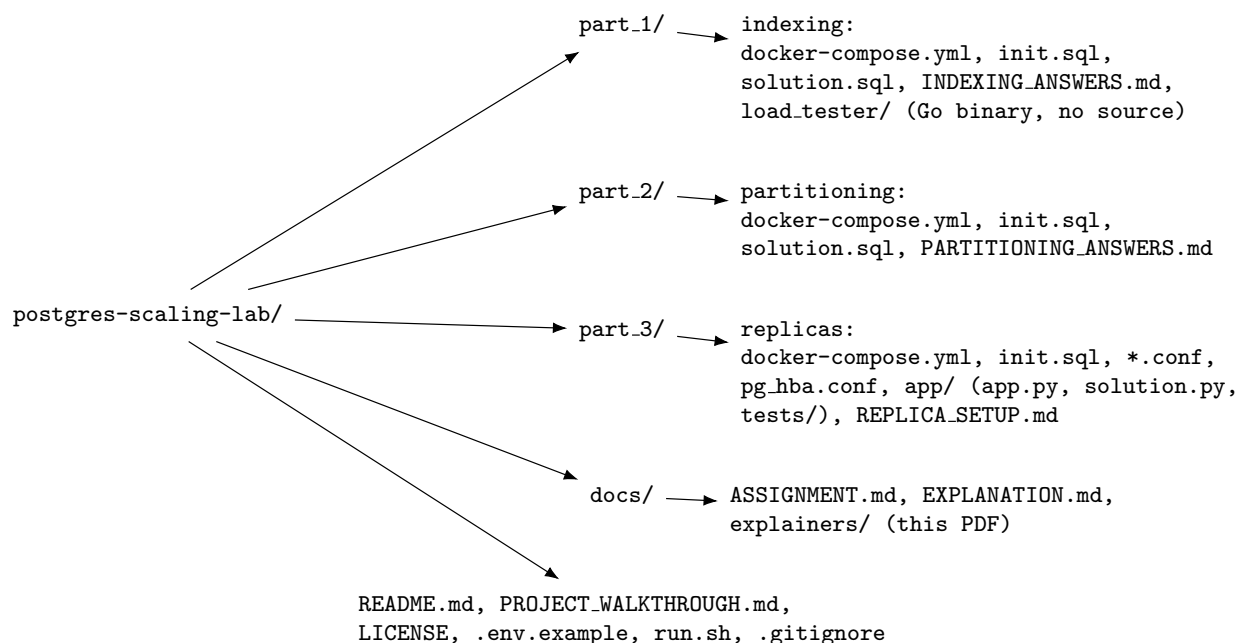
A fourth technique, **sharding** (splitting the data itself across multiple independent database servers, each holding a different subset of rows), is mentioned in the assignment brief as a deliberate non-goal — the brief text literally says “Just kidding, this is not what you will do in this project” — and is not implemented anywhere in this repository. It is mentioned here only because the brief mentions it; no code or configuration for it exists.

1.1 Authorship, honestly

Per the project’s own README, the assignment briefs (`docs/ASSIGNMENT.md`, `part.*/README.md`), the three `init.sql` schemas, the baseline `part.3/app/app.py`, the pytest test suite, and the two prebuilt Go load-tester binaries in `part.1/load_tester/` (`load_tester_amd64`, `load_tester_arm64`, shipped without source) are course-provided scaffolding, not authored by Salman. The author’s own work is the three `solution.sql` files, the entire replication setup in `part.3` (the Docker Compose changes, `primary.conf`, `replica1.conf`, `replica2.conf`, `pg_hba.conf`), the routing applica-

tion `part_3/app/solution.py`, and the analysis documents (`docs/EXPLANATION.md`, the per-part `*_ANSWERS.md` files, and the top-level `README`).

1.2 Repository layout



Each part (`part_1`, `part_2`, `part_3`) is an independent Docker Compose stack with its own database and its own port range; the `README` instructs running `docker compose down` before moving on to the next part so the stacks never collide. **Docker** is a tool that packages an application and everything it needs to run (system libraries, dependencies, configuration) into an isolated unit called a **container**, so it behaves the same on any machine. **Docker Compose** is a companion tool that starts and wires together several containers (here: a database, and sometimes a load generator or a second/third database) from one YAML configuration file.

2 Part 1: Slow-query logging and indexing

2.1 The scenario

The database models a simple e-commerce system: a `users` table (100,000 rows) and an `orders` table (500,000 rows), created by `part_1/init.sql`:

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    email VARCHAR(255),  
    full_name VARCHAR(255),  
    status VARCHAR(20),  
    metadata JSONB,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE orders (  
    id SERIAL PRIMARY KEY,  
    user_id INTEGER, -- foreign key, but NOT indexed by default  
    amount DECIMAL(10,2),  
    status VARCHAR(20),  
    items JSONB,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Both tables are created with *no indexes at all* beyond the automatic one PostgreSQL builds for the **PRIMARY KEY** (a column or set of columns that uniquely identifies each row – PostgreSQL always builds a unique B-tree index for it automatically). Every other lookup must scan the whole table. `users.metadata` and `orders.items` are **JSONB** columns: PostgreSQL’s binary-encoded JSON type, which can store arbitrarily nested documents inside a normal relational column and be queried with special operators such as `@>` (“does this JSON document contain this other JSON document/fragment”).

2.2 Finding the slow queries: slow-query logging and auto_explain

Rather than guessing which queries are slow, the project makes PostgreSQL report on itself. The Compose file (`part_1/docker-compose.yml`) starts the database with:

```
"-c", "log_min_duration_statement=5",  
"-c", "log_statement=none",  
"-c", "shared_preload_libraries=auto_explain",  
"-c", "auto_explain.log_min_duration=5",  
"-c", "auto_explain.log_analyze=on",  
"-c", "auto_explain.log_buffers=on",  
"-c", "auto_explain.log_timing=on"
```

log_min_duration_statement tells PostgreSQL to write a line to its log for every query that takes longer than the given number of milliseconds (here, 5 ms — deliberately low so that even mildly slow queries show up; production systems typically set this to seconds, not milliseconds). **auto_explain** is a PostgreSQL extension that goes one step further: for any query slower than its own threshold, it automatically captures and logs the query’s **EXPLAIN ANALYZE** plan — the step-by-step plan the query planner chose (sequential scan, index scan, nested loop join, etc.),

together with actual timing and, because `log_buffers=on`, the number of 8KB disk buffer blocks touched. This means every slow query shows up in the logs already carrying its own diagnosis, with no separate profiling tool needed.

A second service, `load_tester`, runs continuously alongside the database (`part_1/load_tester/`). It is a prebuilt Go binary shipped in two architectures (`load_tester_amd64`, `load_tester_arm64`, roughly 7–8 MB each) with no source code included in the repository, wrapped in a minimal Dockerfile that just copies the matching binary in and runs it. Its `Makefile` shows how it was originally cross-compiled (`GOOS=linux GOARCH=amd64/arm64 go build`) but gives no insight into what queries it runs. Because the binary is opaque, the README and the top-level project README both describe its behavior as inferred from what appeared in the logs: a mix of queries against `users` and `orders` matching the patterns in the table below (*inferred, not read from source*):

Pattern	Example
Exact match	<code>WHERE email = '...'</code>
Prefix match	<code>WHERE email LIKE 'user%'</code>
Substring match	<code>WHERE full_name ILIKE '%name%'</code>
Range scan	<code>WHERE amount BETWEEN x AND y</code>
Join	<code>users JOIN orders ON users.id = orders.user_id</code>
JSONB containment	<code>WHERE metadata @> '{...}'</code>
Low-selectivity filter	<code>WHERE status = 'active'</code>

2.3 What an index actually is

An **index** is a separate, smaller data structure that PostgreSQL keeps next to a table, storing a sorted or otherwise organized reference from column values to the physical rows that have them. Without one, answering “find the row where `email = 'x'`” requires a **sequential scan**: reading every single row of the table and checking each one, which costs time proportional to the table’s size ($O(n)$). With the right index, PostgreSQL can jump almost directly to the matching row(s) instead.

B-tree. The default and most common PostgreSQL index type. A **B-tree** (“balanced tree”) keeps values in sorted order across a tree of pages, so that finding any value takes about $\log_2(n)$ steps instead of n — for 100,000 rows, roughly 17 comparisons instead of up to 100,000. B-trees are excellent for exact matches (`=`), ranges (`BETWEEN`, `<`, `>`), sorting (`ORDER BY`), and — with the right configuration — prefix matches. They cannot help with a pattern that has a wildcard *before* the fixed part, such as `LIKE '%gmail.com'` or `LIKE '%smith%'`, because a sorted tree of whole values gives no way to jump to “things containing this substring somewhere in the middle.”

GIN (Generalized Inverted Index). Instead of indexing whole values, a **GIN** index stores a list of smaller components (words, array elements, JSON keys) and, for each one, which rows contain it — essentially inside-out (“inverted”) compared to a normal index. This is what makes JSONB containment (`@>`) and full-text/array search possible without scanning every row: a query for `metadata @> {"theme": "dark"}` can go straight to the GIN entry for the key/value pair `theme=dark` and get back exactly the matching row IDs.

pg_trgm and trigram matching. `pg_trgm` is a PostgreSQL extension (enabled in `init.sql` with `CREATE EXTENSION IF NOT EXISTS pg_trgm;`) that breaks text into overlapping 3-character

sequences called **trigrams** (“John” → {“ jo”, “joh”, “ohn”, “hn ”}, roughly). A GIN index built with the `gin_trgm_ops` operator class indexes these trigrams, so a query like `full_name ILIKE '%smith%'` can find rows whose trigram set overlaps with the search term’s trigrams, without a full table scan – something a plain B-tree fundamentally cannot do for infix patterns.

Operator classes matter more than index type. A recurring point in this project: a B-tree index built the “default” way only supports exact-match and range comparisons, not `LIKE 'prefix%'`. To make prefix matching index-friendly, the index has to be built with the `text_pattern_ops` operator class, which stores the text using raw byte ordering instead of the locale-aware collation PostgreSQL normally uses for text – byte order is what lets the planner treat `LIKE 'user%'` as “all values between ‘user’ and ‘user’ and its last-possible successor.” This is a specific, non-obvious decision the solution makes, not a generic “add an index and move on.”

2.4 The solution: `part_1/solution.sql`

```
CREATE INDEX idx_users_email ON users(email text_pattern_ops);
CREATE INDEX idx_users_name_trgm ON users USING GIN(full_name gin_trgm_ops);
CREATE INDEX idx_orders_amount ON orders(amount);
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE INDEX idx_users_metadata_gin ON users USING GIN(metadata);
CREATE INDEX idx_users_status ON users(status);
CREATE INDEX idx_users_created_desc ON users(created_at DESC);
```

Seven indexes, each targeting a specific pattern observed in the slow-query logs:

1. `idx_users_email` (B-tree, `text_pattern_ops`): serves both exact-match email lookups and `LIKE 'prefix%'` searches from the same index.
2. `idx_users_name_trgm` (GIN + `pg_trgm`): serves `ILIKE '%name%'` substring search on `full_name`, which no B-tree configuration can serve.
3. `idx_orders_amount` (plain B-tree): serves `BETWEEN`/range filters on `amount`.
4. `idx_orders_user_id` (plain B-tree): the foreign key from `orders` to `users`. PostgreSQL does *not* automatically index foreign key columns – a common surprise – so joining the two tables without this index means, for every user row, scanning the entire `orders` table looking for matches (a nested loop over a sequential scan). This single index is called out in both README and analysis documents as likely the single highest-impact index of the seven.
5. `idx_users_metadata_gin` (GIN, default JSONB operator class): serves `metadata @> '{...}'` containment queries.
6. `idx_users_status` (plain B-tree): supports filtering by `status`, even though the column only has two distinct values (`active/inactive`) — a **low-cardinality** column, meaning it has very few distinct values relative to the row count. The author’s own answer document (`INDEXING_ANSWERS.md`) and the project’s “what I’d do differently” notes flag this one honestly as marginal: only a ~30% improvement, within the range that could be measurement noise, and its write-time cost (every insert/update must also update this index) was never separately measured.
7. `idx_users_created_desc` (B-tree, descending): supports “newest users first” queries (`ORDER BY created_at DESC`) by storing rows pre-sorted in the exact order the query wants them,

so no separate sort step is needed at query time.

2.5 Measured results

All of the following numbers come from the original Docker run during the course, against PostgreSQL 17.0, as recorded in `README.md` and `part_1/INDEXING_ANSWERS.md`. They are specific to this workload and data distribution, not general performance claims.

Query	Before	After	Improvement	Index used
Email, exact match	35 ms	0.5 ms	98%	<code>idx_users_email</code>
Email, prefix (<code>LIKE 'user%'</code>)	125 ms	3 ms	97%	<code>idx_users_email</code>
Name substring (<code>%name%</code>)	310 ms	180 ms	42%	<code>idx_users_name_trgm</code>
Amount range	3,100 ms	1,300 ms	58%	<code>idx_orders_amount</code>
User/order join	4,500 ms	1,800 ms	60%	<code>idx_orders_user_id</code>
JSONB containment (<code>@></code>)	780 ms	280 ms	64%	<code>idx_users_metadata_gin</code>
Status filter	650 ms	450 ms	30%	<code>idx_users_status</code>

The name-substring result is the most honest limit in this part: even with the trigram index in place, it only improved from 310 ms to 180 ms (42%), the smallest improvement of the seven, because trigram matching still has to check a relatively large set of candidate rows whose trigrams merely overlap the search term — it narrows the search, it does not make it free.

What was deliberately *not* indexed. `SELECT COUNT(*)` queries were left alone: because of PostgreSQL’s **MVCC** (Multi-Version Concurrency Control – the mechanism that lets many transactions read and write concurrently without locking each other out, by keeping multiple versions of each row), counting rows still requires checking each row version’s visibility to the current transaction, so an index does not remove that cost. Similarly, rarely executed queries and low-selectivity multi-column filters (where a large fraction of the table would match anyway) were left un-indexed, since an index that is rarely used but must still be updated on every write is a net cost, not a benefit.

3 Part 2: Range partitioning for time-series data

3.1 Why indexing alone stops helping

Part 2 introduces an `events` table (`part_2/init.sql`) with 600,000 rows: 500,000 spread across the first 358 days of 2024, plus 100,000 more concentrated into the last 14 days to simulate a recent traffic spike:

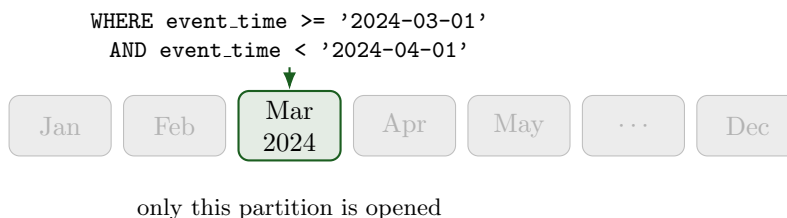
```
CREATE TABLE events (  
    id SERIAL PRIMARY KEY,  
    event_time TIMESTAMP NOT NULL,  
    user_id INTEGER NOT NULL,  
    event_type VARCHAR(50) NOT NULL,  
    event_data JSONB  
);  
  
CREATE INDEX idx_events_time ON events(event_time DESC);  
CREATE INDEX idx_events_time_type ON events(event_time, event_type);  
CREATE INDEX idx_events_user_time ON events(user_id, event_time);
```

This table already ships with three indexes covering the query patterns the assignment brief describes. The point of Part 2 is that indexes alone are not enough for very large time-series tables, for reasons the project's own README explains: a single index over an entire year's data is itself large and must be traversed even for a query that only wants the last 24 hours; hot (recent) and cold (historical) data compete for the same limited memory cache (PostgreSQL's **buffer cache** – an in-memory pool of recently used disk pages); and every write to any row, however old or new, updates the same monolithic index structure.

3.2 What table partitioning is

Table partitioning physically splits one logical table into several smaller physical tables (**partitions**), each holding a subset of the rows, while PostgreSQL still presents them to queries as a single table. **Range partitioning** assigns rows to partitions based on which range of values a chosen column (the **partition key**) falls into – ideal for time-series data, where the natural choice of partition key is the timestamp column itself. (Two other partitioning strategies exist in PostgreSQL – **list** partitioning, for categorical values like region or status, and **hash** partitioning, for spreading rows evenly with no natural range – but neither is used here.)

The payoff is **partition pruning**: when a query's `WHERE` clause restricts the partition key to a known range, the query planner can statically determine which partitions could not possibly contain a match and skip reading them entirely, before execution even begins.



This is exactly the mechanism behind the project's headline partitioning result: a monthly analysis query that read 9,154 buffer blocks (8 KB disk pages) against the unpartitioned table read only 975

blocks against the partitioned one, because 11 of the 12 monthly partitions were never opened at all.

3.3 Mixed-granularity partitioning strategy

The solution (`part_2/solution.sql`) creates a new parent table, `events_partitioned`, and populates it with a mix of partition sizes chosen to match how the data is actually accessed:

```
CREATE TABLE events_partitioned (
    id SERIAL,
    event_time TIMESTAMP NOT NULL,
    user_id INTEGER NOT NULL,
    event_type VARCHAR(50) NOT NULL,
    event_data JSONB
) PARTITION BY RANGE (event_time);

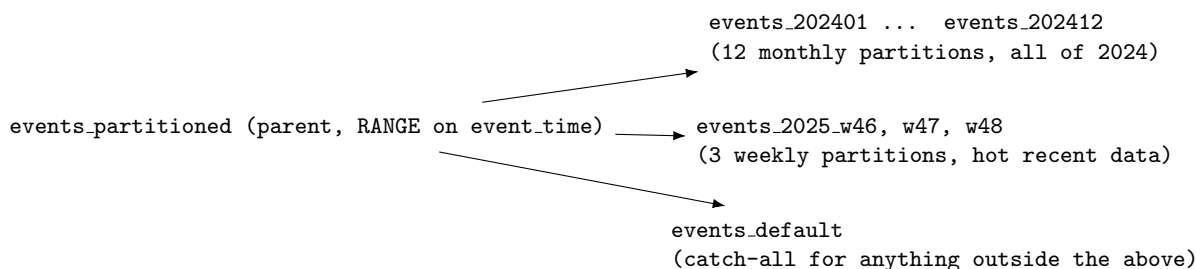
CREATE TABLE events_202401 PARTITION OF events_partitioned
FOR VALUES FROM ('2024-01-01') TO ('2024-02-01');
-- ... one partition per month through events_202412 ...

CREATE TABLE events_2025_w46 PARTITION OF events_partitioned
FOR VALUES FROM ('2025-11-10') TO ('2025-11-17');
-- ... events_2025_w47, events_2025_w48 ...

CREATE TABLE events_default PARTITION OF events_partitioned DEFAULT;

CREATE INDEX idx_p_events_time ON events_partitioned(event_time DESC);
CREATE INDEX idx_p_events_time_type ON events_partitioned(event_time, event_type);
CREATE INDEX idx_p_events_user_time ON events_partitioned(user_id, event_time);

INSERT INTO events_partitioned SELECT * FROM events;
```



Older, less frequently queried 2024 data is split into 12 coarse monthly partitions – fine for historical reporting, simple to maintain. The most recent, most heavily queried data is split into finer weekly partitions, so that a query touching “this week’s” data reads and caches the smallest possible slice. Everything else – including any timestamp value not covered by an explicit range – falls into a **DEFAULT partition**, without which any insert whose `event_time` did not fall inside an existing range would simply fail with an error.

Why the parent table has no PRIMARY KEY. In PostgreSQL, any unique constraint on a partitioned table (including a primary key) must include the partition key as part of the constraint, because uniqueness has to be enforceable per-partition without a global index across all of them.

Since `id` alone is not meaningful as a uniqueness guarantee across partitions here, the parent table simply omits the constraint – a deliberate, documented trade-off, not an oversight.

3.4 Boundary semantics

Partition ranges in PostgreSQL are inclusive on the lower bound and exclusive on the upper bound: `FOR VALUES FROM ('2024-03-01') TO ('2024-04-01')` includes every timestamp from 2024-03-01 00:00:00 up to (but not including) 2024-04-01 00:00:00 – so 2024-03-31 23:59:59.999999 belongs to March, and 2024-04-01 00:00:00 belongs to April, with no ambiguity or double-counting at the edges.

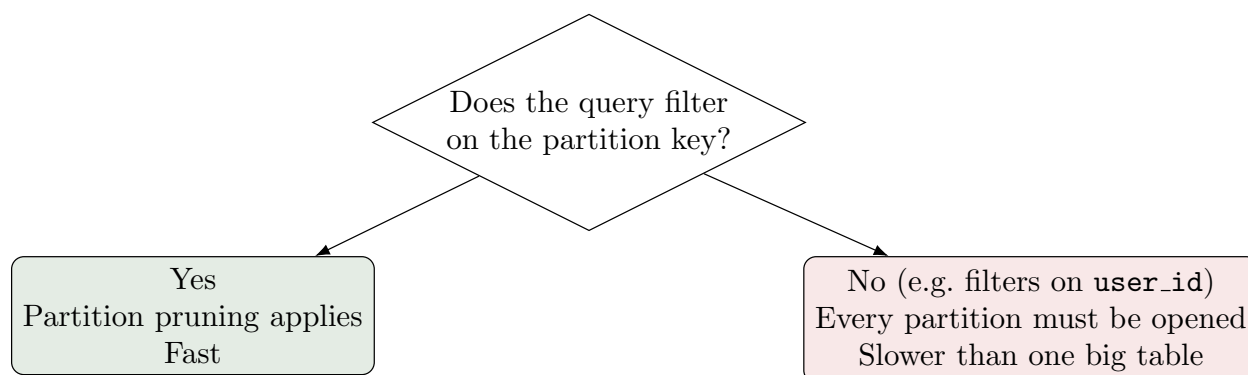
3.5 Measured results, including the honest negative one

Query	Before	After	Improvement
Recent events (last 24h)	1.36 ms	0.70 ms	48%
Monthly analysis (March)	140 ms	86 ms	38%
User activity (last 30 days)	93 ms	160 ms	–70% (slower)
Time range scan (single user, January)	0.06 ms	0.09 ms	≈ 0%

The buffer-block comparison for the monthly analysis query is the steadier and more architecturally meaningful number, because it does not drift with how warm the operating system’s page cache happens to be at the moment of measurement the way wall-clock milliseconds do: 9,154 blocks read before partitioning, versus 975 blocks after – a 90% reduction, because 11 of 12 monthly partitions are never touched.

The user-activity query got 70% slower, and the project kept that result rather than hiding it. The query in question filters on `user_id`, not on `event_time`:

```
SELECT user_id, event_type, COUNT(*)
FROM events
WHERE event_time >= NOW() - INTERVAL '30 days'
GROUP BY user_id, event_type
HAVING COUNT(*) > 100;
```



Even though the query *also* has a 30-day time filter, PostgreSQL still has to open and probe the local index of every partition that overlaps that 30-day window (and, with weekly partitions in the mix, that is several partitions) instead of doing one lookup against one combined index the way

the unpartitioned table allowed. The 30% time filter narrows the data but does not eliminate the per-partition overhead. The project’s own analysis is explicit about the lesson: partitioning is a bet on the access pattern, and this query is the losing side of the bet – choosing `event_time` as the partition key optimizes time-range scans and does nothing for (in fact, actively penalizes) lookups keyed on something else.

3.6 A known limitation left in the code, disclosed rather than hidden

The project’s own “what I’d do differently” notes (top-level `README.md`) flag two specific issues in `part_2/solution.sql` that were not fixed:

- The partitions are **hardcoded** to specific calendar dates (monthly ranges for 2024, weekly ranges for a specific week of November 2025), while `init.sql` generates “recent” data relative to whatever `NOW()` is when the seed script runs. Run today, the “recent” rows would land in `events_default` instead of a purpose-built weekly partition, and the hot-data speedup would not reproduce without updating the partition ranges. A DO block computing boundaries from the actual data (or the `pg_partman` extension) is named as the fix that was not implemented.
- `INSERT INTO events_partitioned SELECT * FROM events` copies every row’s existing `id` value across but never advances the new table’s own `id` sequence, so a fresh insert after the migration could collide with an existing `id`. A `setval()` call after the copy is the documented, un-applied fix.

4 Part 3: Primary plus two streaming read replicas

4.1 The remaining bottleneck

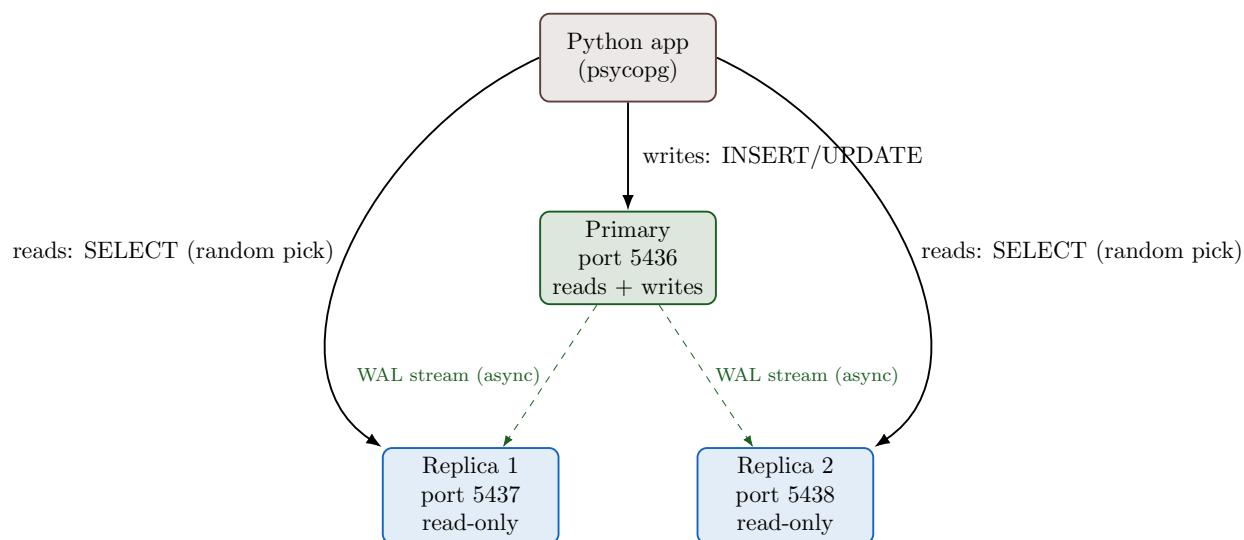
Even with indexes and partitioning, one server still answers every read *and* every write. Part 3 addresses this with **replication**: continuously copying all data from one **primary** database to one or more **replica** databases, and then routing read queries to the replicas instead of the primary, so read traffic (typically the majority of traffic in many applications) no longer competes with writes for the same server's resources.

4.2 WAL and streaming replication

PostgreSQL's replication mechanism is built on the **WAL** (Write-Ahead Log): before any change to the actual data files is made, PostgreSQL first writes a record of that change to a sequential log file. This exists primarily for crash recovery (replay the log to reconstruct lost changes), but the same log can also be streamed continuously to other servers, which replay it themselves to stay in sync – this is **streaming replication**. A replica that continuously replays WAL from a primary is called a **standby**; because `hot_standby = on` is set here, the standbys are also allowed to serve read-only queries *while* replaying, rather than being replay-only and unreadable.

Replication here is **asynchronous**: the primary commits a transaction and returns success to the client without waiting for any replica to confirm it received the change (`synchronous_commit = off` in `primary.conf`). This keeps writes fast, at the cost of a small window in which a replica's data can lag slightly behind the primary's – covered in detail below.

4.3 Cluster topology



Each replica is built by taking a **base backup** of the primary – `pg_basebackup`, a PostgreSQL tool that copies a full, consistent snapshot of the primary's data directory to a new server – and then continuously streaming and replaying WAL from that point forward, using a **physical replication slot** (`replica1_slot`, `replica2_slot`, created in `init.sql` with `pg_create_physical_replication_slot(...)`), which reserves WAL on the primary so it is not discarded before a possibly-slower replica has consumed it.

4.4 Configuration walkthrough

`primary.conf` sets `wal_level = replica` (generate enough detail in the WAL for replicas to reconstruct changes – a lower level would omit information replicas need), `max_wal_senders = 10` (up to 10 concurrent replication connections), `max_replication_slots = 10`, `synchronous_commit = off` (asynchronous, as above), and `wal_keep_size = 1GB` (retain at least this much WAL so a temporarily disconnected replica can catch up without needing a fresh base backup). It also points `hba_file` at a custom `pg_hba.conf`.

`pg_hba.conf` (“host-based authentication” – the file controlling which clients may connect, from where, as whom, and how) is deliberately permissive for a lab environment: it trusts all connections, including replication connections, from any address (`0.0.0.0/0`) with the `trust` method (no password check at the network layer at all). This is appropriate only because the whole cluster runs on an isolated Docker network with no external exposure beyond the mapped host ports; it would not be a safe configuration on a network-reachable production server.

`replica1.conf` / `replica2.conf` each set `hot_standby = on` (allow read queries during replication), `primary_conninfo` (how to connect back to the primary, including a distinguishing `application_name` of `replica1` / `replica2` so `pg_stat_replication` on the primary can tell them apart), and `primary_slot_name` matching the slot created for that replica. `hot_standby_feedback = on` tells the primary about queries currently running on the replica, so the primary avoids cleaning up (**vacuuming**) row versions the replica’s long-running read queries might still need – without it, a long read on a replica could be cancelled out from under it by the primary’s routine cleanup.

`docker-compose.yml` wires the three services together. Each replica’s startup command is a small shell script, not just a plain `postgres` invocation:

```
echo "primary:5432:*:${POSTGRES_USER}:${POSTGRES_PASSWORD}" > /var/lib/postgresql/.pgpass
chmod 600 /var/lib/postgresql/.pgpass
if [ ! -s /var/lib/postgresql/data/PG_VERSION ]; then
    pg_basebackup -h primary -U postgres -p 5432 -D /var/lib/postgresql/data -Fp -Xs -P -R
    chmod 700 /var/lib/postgresql/data
fi
postgres -c config_file=/etc/postgresql/postgresql.conf
```

This generates a `.pgpass` file (PostgreSQL’s mechanism for supplying a password to a command-line tool with no interactive terminal to type one into) at container startup, from environment variables, then only runs `pg_basebackup` the first time the container starts (detected by checking whether `PG_VERSION`, a marker file PostgreSQL writes once its data directory is initialized, already exists) – so a replica that already has data from a previous run does not re-copy the whole primary on every restart.

4.4.1 A committed secret, found and fixed

The project’s own challenges section documents a real issue that was caught and corrected: the original part 3 setup checked a plaintext credentials file, `part_3/.pgpass`, containing `primary:5432:*:postgres:postgres` directly into the repository (visible in `part_3/diff.patch`, which is the retained record of that original change). That file existed because `pg_basebackup -R` needs a password with no terminal available to prompt for one. The fix applied was to stop committing the file and instead generate

it at container boot, as shown above – the credential still reaches `pg.basebackup` non-interactively, but no copy of it lives in version control anymore. (In this lab context the password is the literal string `postgres`, overridable via `POSTGRES_PASSWORD`; the point of the fix is the practice of not committing secrets, not the sensitivity of this specific lab password.)

4.5 The routing application: `app.py` versus `solution.py`

The course-provided baseline, `part_3/app/app.py`, is intentionally simple and un-scaled: every method – writes and reads alike – opens a connection to the exact same single database URL (`DATABASE_URL`, defaulting to one PostgreSQL instance on port 5432). There is no read/write distinction at all.

The author's solution, `part_3/app/solution.py`, replaces the single connection string with two: a primary DSN (**DSN**, Data Source Name – a connection string identifying a specific database server, e.g. `postgresql://user:pass@host:port/db`) for writes, and a list of replica DSNs for reads, both configurable via environment variables (`PRIMARY_DSN`, `REPLICA_DSNS`) with defaults matching the Compose stack's ports (5436 for the primary, 5437/5438 for the two replicas):

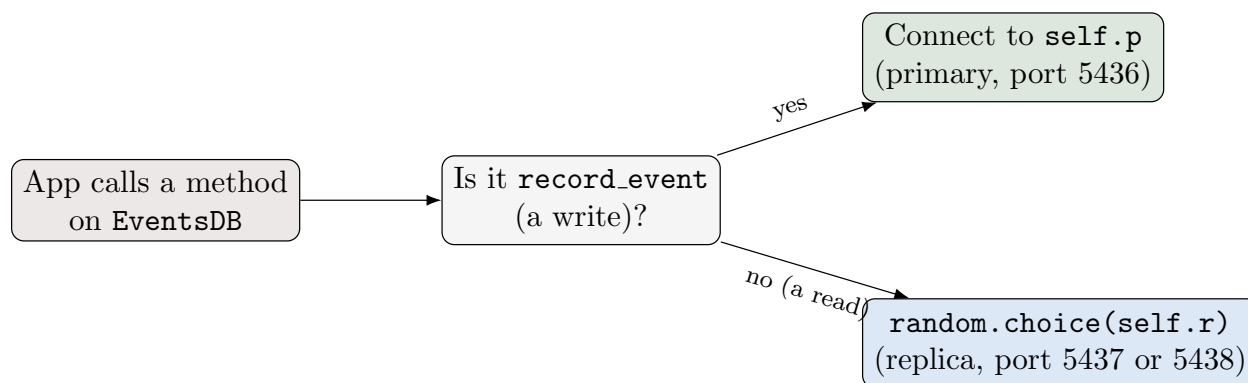
```
class EventsDB:
    def __init__(self):
        self.p = os.environ.get("PRIMARY_DSN", ".../5436/events_db")
        self.r = os.environ.get("REPLICA_DSNS",
                                ".../5437/events_db,.../5438/events_db").split(",")

    def record_event(self, event_type, user_id, metadata=None):
        # write -- always the primary
        with psycopg.connect(self.p) as conn:
            ...

    def get_recent_events(self, hours=24):
        # read -- a randomly chosen replica
        conn_str = random.choice(self.r)
        with psycopg.connect(conn_str) as conn:
            ...

    # get_user_activity and get_monthly_stats follow the same read pattern
```

psycopg is the PostgreSQL driver library for Python used here (version 3) – the code that actually opens a network connection to PostgreSQL and speaks its wire protocol so Python code can run SQL against it. Every one of the three read methods (`get_recent_events`, `get_user_activity`, `get_monthly_stats`) picks one replica DSN uniformly at random with `random.choice()` for each call – a simple, stateless form of **load balancing** (spreading requests across multiple servers). Every write method (`record_event`) always targets `self.p`, the primary, unconditionally.



4.6 Replication lag and the “read-your-writes” problem

Because replication is asynchronous, a row written to the primary is not guaranteed to be visible on a replica the instant the write returns – there is a small window (typically low milliseconds under light load) during which the WAL for that write is still in flight or being replayed. If a read query happens to hit a replica during that window, it will not see the row yet. This class of problem is known as **read-your-writes** consistency (or its absence): a client cannot assume it will immediately see its own just-written data if reads and writes are not guaranteed to go to the same, fully-caught-up server.

The project does not attempt to solve this in the application layer (e.g. with causal-consistency tokens or routing a read back to the primary right after a write); it is left as async, and the test suite (`part_3/app/tests/test_events_db.py`) instead *tolerates* it explicitly, inserting a short `time.sleep(0.5)` after every write before reading, and further guarding several assertions with `if events: / if len(...) > 0`: checks so that a read that still hasn’t seen the replicated row does not fail the test outright. The project’s own “what I’d do differently” section is candid that this makes the tests capable of passing *vacuously* – a completely broken replication path could, in principle, still show green, because the assertions are skipped rather than enforced when no data has arrived yet. A polling loop with a timeout is named as the more rigorous alternative that was not implemented.

4.7 Test suite and what it demonstrated

```

def test_get_user_activity(events_db):
    test_user_id = random.randint(10000, 99999)
    events_db.record_event("test_event_1", test_user_id)
    events_db.record_event("test_event_2", test_user_id)
    time.sleep(0.5) # allow for replication lag
    user_activity = events_db.get_user_activity(test_user_id)
    assert isinstance(user_activity, list)
    if len(user_activity) > 0:
        for activity in user_activity:
            assert isinstance(activity, tuple)
            assert len(activity) == 2
  
```

Five tests exist in total: recording an event with metadata, recording one without metadata, reading recent events, reading a single user’s activity, and reading monthly aggregate stats. Each generates a random user id or event-type prefix so concurrent test runs cannot collide with each other’s

data. Per the project's own documentation, all five passed (**5 passed in 2.41s**) when run against a local PostgreSQL instance with every DSN (`PRIMARY_DSN` and both `REPLICA_DSNS`) pointed at the same single server – i.e. the routing *logic* and query correctness were verified, but not against an actual multi-node replicated cluster in this environment (see the verification section below for exactly what was and was not exercised).

5 Verification status — what was actually run, stated plainly

This section exists because the one rule that outranks every other claim in this project is: nothing should be claimed that was not actually verified. Here is exactly what was and was not done, per the project’s own README:

- **SQL validated on PostgreSQL 18.** Every `init.sql` and `solution.sql` file across all three parts was executed against a real PostgreSQL 18 server and ran without error, confirming the SQL itself (schema creation, index creation, partition creation, data migration) is syntactically and semantically correct on a current PostgreSQL release — one version newer than the 17.0 used for the original benchmark numbers above.
- **The Part 3 routing app passed its full test suite (5/5)** against a local PostgreSQL instance, with `PRIMARY_DSN` and `REPLICA_DSNS` all pointed at that one server. This confirms the read/write routing *code path* works correctly (writes use the primary DSN, reads use a replica DSN, queries return correctly shaped results) but does *not* by itself confirm behavior against a real multi-node streaming-replication cluster with genuine replication lag, since only one physical server was involved in that run.
- **All three docker-compose.yml files pass docker compose config**, meaning each is syntactically valid Compose YAML that resolves cleanly (environment variable substitution, volume and network definitions, service dependencies) – but this only validates the *configuration file*, not that the containers actually start and behave correctly.
- **The full Docker stacks were *not* brought up live** on the machine used for this latest round of verification, because the Docker daemon was not running there. This means none of the three `docker compose up` stacks – including the primary/replica cluster in Part 3 – were actually started, observed for successful `pg_basebackup` completion, checked via `pg_stat_replication` for genuinely streaming replicas, or exercised end-to-end against live replication lag in this environment. The benchmark numbers quoted throughout this document (35 ms → 0.5 ms, 9,154 → 975 buffer blocks, etc.) come from the original Docker runs performed during the course, not from a fresh run in this verification pass.

In short: the logic, SQL, and configuration are confirmed correct as far as static validation and single-node testing can confirm; the live, multi-container, actually-replicating behavior of the Part 3 cluster was not re-observed in this pass, and this document does not claim otherwise.

6 Challenges encountered, in the author’s own words

The following five points are carried over directly (condensed, not altered in substance) from the project’s own README “Challenges” section, because they are the most concrete evidence of engineering judgment in the project and should not be paraphrased into something vaguer:

1. The user-activity query got slower after partitioning (documented above, Part 2), and the result was kept and explained rather than hidden or re-benchmarked away.
2. A plaintext `.pgpass` credentials file was committed to the repository and later removed in favor of generating it at container boot (documented above, Part 3).
3. Reads can return stale data (missing a just-written row) because of asynchronous replication lag; there is no way around this without either synchronous replication or read-your-writes

logic in the application, and the project chose to stay async and be honest about the resulting eventual consistency rather than add complexity to hide it.

4. A plain B-tree index answered almost none of the Part 1 query patterns without additional configuration: exact match works out of the box, but prefix matching needs `text_pattern_ops`, substring matching needs GIN + `pg_trgm`, and JSONB containment needs a GIN index on the JSONB column specifically – each is a genuinely different operator class, not an interchangeable tuning knob.
5. The Part 1 load generator is an opaque, prebuilt Go binary with no included source, so its exact query mix could not be inspected or altered directly; the workaround was to make the database report on itself via `log_min_duration_statement` and `auto_explain`, which is how the before/after timing numbers were captured without ever needing to see inside the load generator.

7 What was learned

Also drawn directly from the project’s own README:

- Partition pruning only benefits queries that filter on the partition key; choosing `event_time` as that key is really a choice about *which queries you are willing to make slower* (anything keyed on something else, like `user_id`).
- The index type/operator class is itself a design decision: `text_pattern_ops` for prefixes, GIN + `pg_trgm` for substrings, GIN for JSONB containment – each exists because a default B-tree simply cannot serve that kind of predicate, regardless of tuning.
- Asynchronous streaming replication buys read scale-out but relocates the consistency problem into the application: once reads can lag behind writes, the application (and its tests) must be written to tolerate staleness – a coupling that does not exist at all on a single node.
- Secrets management is a startup-time concern, not a repository concern: generating `.pgpass` from an environment variable at container boot feeds the same credential to `pg_basebackup` without ever storing it in the source tree.
- Buffer-block counts from EXPLAIN (BUFFERS) are a more stable basis for comparison than wall-clock milliseconds, because they reflect what the query planner actually does (partition pruning, index usage) rather than incidental warmth of the operating system’s cache at measurement time.

8 What the author would do differently

- Generate Part 2’s partitions programmatically (a DO block computing boundaries from the actual data, or the `pg_partman` extension) instead of hardcoding fixed calendar dates that silently stop matching “recent” data as time passes.
- Call `setval()` after migrating rows into `events_partitioned` so the new table’s sequence does not collide with copied-over ids.
- Use connection pooling (e.g. `psycopg_pool`) in Part 3 instead of opening a fresh connection per query, which is the dominant per-call latency cost in both `app.py` and `solution.py`.

- Make replica selection health-aware instead of blind `random.choice()` over two DSNs, where one dead replica currently fails 50% of reads; even a naive retry against the other replica would help, and a production setup would put something like PgBouncer or HAProxy in front of the replicas.
- Rewrite the Part 3 tests to poll (with a timeout) until replicated data actually appears, instead of a fixed sleep plus conditionally-skipped assertions that can pass even when replication is completely broken.
- Keep only the Part 1 indexes that clearly earn their place (the status-column index's ~30% gain is within noise) and measure write-side cost (insert latency) before and after, which was never done here.

9 Appendix: file-by-file quick reference

File	Role
README.md	Top-level summary, results tables, setup instructions, challenges, lessons, authorship note. Author-written.
docs/ASSIGNMENT.md	Original course assignment brief. Course-provided.
docs/EXPLANATION.md	Full narrative technical write-up with additional diagrams. Author-written.
part_1/init.sql	Creates <code>users/orders</code> , seeds 100k/500k rows, enables <code>pg_trgm</code> . Course-provided.
part_1/solution.sql	The 7 indexes described above. Author-written.
part_1/INDEXING_ANSWERS.md	Author's own analysis of the 7 indexes and results. Author-written.
part_1/load_tester/	Prebuilt Go binaries (amd64/arm64) simulating traffic; Dockerfile and Makefile only, no Go source. Course-provided binaries.
part_2/init.sql	Creates unpartitioned <code>events</code> table with 3 indexes, seeds 600k rows. Course-provided.
part_2/solution.sql	Creates <code>events_partitioned</code> with 12 monthly + 3 weekly + 1 default partition, migrates data. Author-written.
part_2/PARTITIONING_ANSWERS.md	Author's own before/after analysis, including the negative result. Author-written.
part_3/init.sql	Creates replication slots, <code>events</code> table, seed data. Course-provided (slot creation added for replication).
part_3/primary.conf	Primary server config (WAL level, async commit, wal senders/slots). Author-written.
part_3/replica1.conf, replica2.conf	Per-replica standby config (<code>hot_standby</code> , <code>primary_conninfo</code> , slot name). Author-written.
part_3/pg_hba.conf	Trust-based auth rules for the isolated lab network. Author-written.
part_3/docker-compose.yml	Wires primary + 2 replicas, base-backup-on-first-boot logic, <code>.pgpass</code> generation. Author-written.

File	Role
<code>part_3/diff.patch</code>	Retained diff showing the original (insecure) setup and the fix, for transparency. Author-written/retained.
<code>part_3/app/app.py</code>	Baseline single-connection <code>EventsDB</code> , no read/write split. Course-provided.
<code>part_3/app/solution.py</code>	Read/write-routing <code>EventsDB</code> (primary for writes, random replica for reads). Author-written.
<code>part_3/app/tests/test_eventsdb.py</code>	Unit/pytest suite exercising both write and read paths. Course-provided.
<code>part_3/REPLICA_SETUP.md</code>	Author's own short summary of the replica setup. Author-written.
<code>.env.example</code>	Documents every environment variable consumed across all three parts. Author-written.
<code>run.sh</code>	One combined script chaining all three parts' commands end-to-end for a full demo run. Author-written.